SECURE SHELL ASSIGNMENT

Task 1: Install OpenSSH Server and verify SSH

Step 1: Open the Linux terminal

Log in to your RHEL Linux machine and switch to the root user.

su -

Enter your root password when prompted.

Step 2: Install the OpenSSH server package

Run:

dnf install openssh-server -y

Verify that it is installed:

rpm -q openssh-server

You should see the installed package name and version.

Step 3: Start and enable the SSH service

Run these commands one by one:

systemctl enable --now sshd

Check the service status:

systemctl status sshd

Look for:

Active: active (running)

Press q to exit the status screen.

Step 4: Test SSH by connecting to localhost

Run:

ssh localhost

If prompted to trust the host, type yes. Enter your Linux user's password if requested.

After successful login, verify your identity:

whoami

Then exit the SSH session:

exit

Task 1 is complete when the package is installed, sshd is running, and you can connect to localhost using SSH.

Task 2: Generate SSH keys and enable key-based authentication

Step 1: Switch to your normal user account

If you are currently root, switch to your user. I'll use diya as an example; replace it with your actual username if different.

su - diya

Verify your username:

whoami

Expected output:

diya

Step 2: Generate a new SSH key pair

Run:

ssh-keygen -t ed25519 -C "diya@server"

When prompted:

- File to save the key: Press Enter to accept the default location, /home/diya/.ssh/id_ed25519.
- Passphrase: Enter a passphrase if desired, or press Enter twice for no passphrase in this lab.

Two files will be generated:

File	Purpose
~/.ssh/id_ed25519	Private key — keep it secret
~/.ssh/id_ed25519.pub	Public key — added to authorized_keys

Verify the files:

ls -l ~/.ssh/

Step 3: Add the public key to authorized_keys

Create the SSH directory if necessary and set the correct permissions:

mkdir -p ~/.ssh

chmod 700 ~/.ssh

Add your public key:

cat ~/.ssh/id_ed25519.pub >> ~/.ssh/authorized_keys

Set the permissions:

chmod 600 ~/.ssh/authorized_keys

Verify that the public key is present:

cat ~/.ssh/authorized_keys

You should see a long line beginning with ssh-ed25519.

Step 4: Test key-based login

Run:

ssh -o PreferredAuthentications=publickey -o PasswordAuthentication=no diya@localhost

Replace diya with your username if needed.

If the login succeeds without asking for your Linux account password, key-based authentication is working. If you set a private-key passphrase, SSH may ask for that passphrase instead.

Verify and exit:

whoami

exit

Task 2 is complete when you have generated the key pair, added the public key to authorized_keys, and successfully logged in using the key.

Task 3: Disable password authentication

Important: Do not close your existing terminal session until you have verified that key-based login works after the configuration change.

Step 1: Open the SSH server configuration file

Switch to root:

su -

Open the configuration file using vi:

vi /etc/ssh/sshd_config

Find the line containing PasswordAuthentication.

In vi, type:

/PasswordAuthentication

Press Enter to search. Press n to find the next match if required.

Make sure the effective configuration contains:

PubkeyAuthentication yes

PasswordAuthentication no

If the setting is commented out or set to yes, edit it so that the active setting is PasswordAuthentication no. Also check for duplicate active settings elsewhere in the file.

To edit in vi:

1. Press i to enter insert mode.
2. Make the required changes.
3. Press Esc.
4. Type :wq and press Enter to save and exit.

Step 2: Validate the configuration

Run:

sshd -t

If there is no output, the configuration syntax is valid.

If an error appears, correct it before restarting the service.

Step 3: Restart the SSH service

systemctl restart sshd

Verify the service:

systemctl status sshd

Look for Active: active (running).

Step 4: Test key-based login again

From your normal user account, run:

ssh -o PreferredAuthentications=publickey -o PasswordAuthentication=no diya@localhost

Confirm that you can log in successfully.

Step 5: Verify that password login is disabled

Exit the SSH session if necessary, then run this from your normal user account:

ssh -o PubkeyAuthentication=no -o PreferredAuthentications=password diya@localhost

Expected result: authentication fails, typically with:

Permission denied

This test deliberately disables public-key authentication for the client, so it checks that password-based login is not available. If the connection still succeeds, check the effective SSH configuration and any included configuration files.

You can inspect the effective settings with:

sudo sshd -T | grep -E '^(passwordauthentication|pubkeyauthentication) '

Expected output:

pubkeyauthentication yes

passwordauthentication no

Task 3 is complete when SSH key login succeeds and password authentication fails.

Task 4: Document the procedure for your assignment

You can copy the following report into your assignment document and add your own screenshots and actual command outputs.

Configuration of SSH Key-Based Authentication in Linux

1. Aim

To install the OpenSSH server, configure SSH key-based authentication, and disable password-based authentication on a Linux system.

2. Requirements

- RHEL 9 Linux system
- Root or sudo privileges
- OpenSSH server package
- Linux user account

3. Procedure

Step 1: Install OpenSSH Server

Command:

dnf install openssh-server -y

Verification:

rpm -q openssh-server

Step 2: Start and Enable SSH Service

Commands:

systemctl enable --now sshd

systemctl status sshd

The SSH service was checked to ensure that it was active and running.

Step 3: Test Local SSH Connection

Command:

ssh localhost

The connection was tested using the local Linux system.

Step 4: Generate an SSH Key Pair

Command:

ssh-keygen -t ed25519 -C "diya@server"

The command generated a private key and a public key in the user's .ssh directory.

Step 5: Configure Authorized Keys

Commands:

mkdir -p ~/.ssh

chmod 700 ~/.ssh

cat ~/.ssh/id_ed25519.pub >> ~/.ssh/authorized_keys

chmod 600 ~/.ssh/authorized_keys

The public key was added to the authorized_keys file to allow key-based authentication.

Step 6: Test Key-Based Authentication

Command:

ssh -o PreferredAuthentications=publickey -o PasswordAuthentication=no diya@localhost

Successful login confirmed that SSH key-based authentication was working.

Step 7: Disable Password Authentication

Configuration file:

/etc/ssh/sshd_config

Configuration changes:

PubkeyAuthentication yes

PasswordAuthentication no

The SSH configuration syntax was validated using:

sshd -t

The service was restarted using:

systemctl restart sshd

Step 8: Verify Authentication Settings

Command:

sshd -T | grep -E '^(passwordauthentication|pubkeyauthentication) '

The expected effective settings were:

- pubkeyauthentication yes
- passwordauthentication no

Step 9: Test Password Authentication

Command:

ssh -o PubkeyAuthentication=no -o PreferredAuthentications=password diya@localhost

The password-only login attempt was expected to fail, confirming that password authentication was disabled.

4. Result

The OpenSSH server was installed and configured successfully. SSH key-based authentication was enabled, and password-based authentication was disabled. The configuration and authentication methods were verified using SSH commands and service status checks.

5. Conclusion

SSH key-based authentication provides a secure method of remote login without relying on the Linux account password for SSH authentication. Disabling password authentication helps reduce the risk of password-guessing attacks.